MEDICAPS UNIVERSITY

Data Structures in R

Laxmikant Soni

www.medicaps.ac.in

Data Structures in R

R provides a rich collection of **data structures** that allow efficient storage, manipulation, and analysis of data. These structures form the backbone of data handling in R and are designed to support both **statistical computations** and **data analysis workflows**. Below are the primary data structures in R, with short descriptions and examples.

Vector

The most basic data structure in R, storing elements of the same type (numeric, character, logical, etc.).

```
# numeric vector
v_num <- c(1, 2, 3, 4)
# character vector
v_chr <- c("a", "b", "c")
```

Matrix

A two-dimensional, homogeneous data structure where all elements are of the same type.

```
m <- matrix(1:6, nrow = 2, ncol = 3)
print(m)
```

Array

A multi-dimensional generalization of matrices, useful for storing data in more than two dimensions.

```
arr <- array(1:8, dim = c(2, 2, 2))
print(arr)
```

List

A versatile structure that can hold elements of different types, including other lists or data structures.

```
lst <- list(name = "Laxmi", age = 30, scores =
c(90, 85, 88))
str(lst)
```

Data Frame

A tabular structure widely used in data analysis, where each column can be of a different type but all columns have equal length.

```
df <- data.frame(Name = c("A", "B"), Age =
c(23, 25), Score = c(89, 92))
head(df)
```

Factor

A special data structure for handling categorical variables, storing data as integer codes with associated labels.

```
f <- factor(c("Male", "Female", "Male"))
levels(f)
```

Quick Comparison Table

Structure	Dimensions	Homogeneous?	Typical use case
Vector	1D	Yes	Basic sequences of values
Matrix	2D	Yes	Numeric tables, linear algebra
Array	ND	Yes	Multi-dimensional numeric data
List	1D (nested)	No	Heterogeneous collections
Data frame	2D	Columns may differ	Tabular data for analysis
Factor	1D (categorical)	No (internally integers)	Categorical variables

Notes and Tips

- Use **vectors** for atomic homogeneous data.
- Use **lists** when you need to bundle mixed types or complex objects.
- Use **data frames** (or tibbles from the `tibble` package) for tabular data ready for `dplyr`/`tidyr` workflows.
- Convert character columns to **factors** when they represent categorical variables with a fixed set of levels.

Vector Operations

- Arithmetic: `v_num * 2`, `v_num + v_num`
- Indexing: `v_num[2]`
- Logical filtering: `v_num[v_num > 2]`

Vector Operations

- Arithmetic: `v_num * 2`, `v_num + v_num`

```
v_num <- c(1, 2, 3, 4)
v_num * 2          # Multiply each element by 2
v_num + v_num      # Element-wise addition
```

- Indexing: `v_num[2]`

```
v_num[2]  # Returns the second element (2)
```

- Logical filtering: `v_num[v_num > 2]`

```
v_num[v_num > 2]  # Returns elements greater
than 2 (3, 4)
```

Matrix Operations

- Transpose: `t(m)`
- Matrix multiplication: `m %*% t(m)`
- Indexing rows/columns: `m[1, ]`, `m[, 2]`

Matrix Operations

- Transpose: `t(m)`

```
m <- matrix(1:6, nrow = 2, ncol = 3)
t(m)
```

- Matrix multiplication: `m %*% t(m)`

```
m %*% t(m)  # Matrix multiplication
```

- Indexing rows/columns: `m[1, ]`, `m[, 2]`

```
m[1, ]   # First row
m[, 2]   # Second column
```

Array Operations

- Access elements using multi-index: `arr[1,2,1]`
- Apply function across dimensions: `apply(arr, c(1,2), sum)`

Array Operations

- Access elements using multi-index: `arr[1,2,1]`

```
arr <- array(1:8, dim = c(2, 2, 2))
arr[1,2,1]  # Element at (1,2,1)
```

- Apply function across dimensions: `apply(arr, c(1,2), sum)`

```
apply(arr, c(1,2), sum)  # Sums over 3rd dimension
```

List Operations

- Access by index: `lst[[1]]`
- Access by name: `lst$name`
- Append element: `lst$new <- "extra"`

List Operations

- Access by index: `lst[[1]]`

```
lst <- list(name = "Laxmi", age = 30, scores =
c(90, 85, 88))
lst[[1]]  # First element ("Laxmi")
```

- Access by name: `lst$name`

```
lst$name  # Access element by name
```

- Append element: `lst$new <- "extra"`

```
lst$new <- "extra"
str(lst)
```

Data Frame Operations

- Column access: `df$Name`
- Subset rows: `df[df$Age > 23, ]`
- Add new column: `df$Grade <- c("A", "B")`

Data Frame Operations

- Column access: `df$Name`

```
df <- data.frame(Name = c("A", "B"), Age = c(23, 25), Score = c(89, 92))
df$Name
```

- Subset rows: `df[df$Age > 23, ]`

```
df[df$Age > 23, ]  # Rows where Age > 23
```

- Add new column: `df$Grade <- c("A", "B")`

```
df$Grade <- c("A", "B")
head(df)
```

Factor Operations

- Levels: `levels(f)`
- Frequency table: `table(f)`
- Reordering: `factor(f, levels = c("Female", "Male"))`

Factor Operations

- Levels: `levels(f)`

```
f <- factor(c("Male", "Female", "Male"))
levels(f)
```

- Frequency table: `table(f)`

```
table(f)  # Counts of each category
```

- Reordering: `factor(f, levels = c("Female", "Male"))`

```
f2 <- factor(f, levels = c("Female", "Male"))
f2
```